

Seleção de Meta-Features com Algoritmos Genéticos



Projeto Final — Tema 4 — Disciplina de Meta-aprendizagem

Renato Coca Terrazas Freire | UFAL — Instituto de Computação

Professor: Lucas Amorim | Junho de 2026

Sumário

Resumo	3
1. Introdução.....	4
2. Metodologia	5
2.1 Construção do Meta-Dataset 5	
2.2 Extração de Meta-Features 5	
2.3 Algoritmo Genético 5	
2.4 Avaliação 6	
3. Resultados	7
3.1 Performance dos Algoritmos Base 7	
3.2 Baseline do Meta-Modelo 7	
3.3 Convergência do Algoritmo Genético 8	
3.4 Comparação: Baseline vs Após GA 8	
3.5 Teste de Wilcoxon 9	
3.6 Sobreposição: GA vs Importância RF 10	
4. Conclusões	11
5. Referências	12

Resumo

Este trabalho investiga o efeito da seleção de meta-features na performance de sistemas de meta-aprendizagem. Partindo do Exemplo 3 da disciplina (22 datasets, 56 meta-features), o meta-dataset foi expandido para 35 datasets e 224 meta-features, incorporando 5 grupos do pymfe e 4 medidas de resumo. Um Algoritmo Genético (GA) com cromossomo binário foi aplicado para selecionar o subconjunto ótimo de meta-features, reduzindo de 224 para 114 features (-49,1%) com melhora de 9,9% no RMSE do meta-modelo para o algoritmo-alvo. Os resultados mostram que a expansão de meta-features já melhora o Spearman ρ de 0,30 para 0,76 antes do GA, e que o GA realiza seleção mais sofisticada que um simples ranking por importância, eliminando redundâncias entre features correlacionadas.

1. Introdução

Meta-aprendizagem é o paradigma em que um sistema aprende, a partir de experiências passadas em múltiplos datasets, qual algoritmo de aprendizagem terá melhor desempenho em um novo problema. A chave para esse processo são as meta-features: características estatísticas extraídas dos datasets que descrevem sua complexidade, distribuição e estrutura interna.

Um desafio central é a seleção dessas meta-features. Conjuntos muito grandes introduzem ruído e redundância, enquanto conjuntos muito pequenos perdem informação discriminativa. Algoritmos Genéticos são uma abordagem eficaz para esse problema de seleção combinatória, explorando o espaço de subconjuntos possíveis de forma guiada pela evolução.

Este projeto, baseado no Exemplo 3 da disciplina, expande o trabalho base em duas dimensões — mais datasets e mais meta-features — e aplica um GA para otimizar a seleção, respondendo três perguntas de pesquisa:

- (1) A seleção de meta-features via GA melhora a performance do meta-modelo?
- (2) Qual subconjunto de meta-features é selecionado pelo GA?
- (3) As features selecionadas pelo GA são consistentes com a importância apontada pelo Random Forest?

2. Metodologia

2.1 Construção do Meta-Dataset

O meta-dataset foi construído em duas etapas. Na Parte A, os 22 datasets do Exemplo 3 foram obtidos do OpenML (20 encontrados; usps e pizzacutter3 indisponíveis publicamente). Na Parte B, 15 datasets adicionais foram coletados da benchmark suite OpenML-CC18 (72 tasks), filtrando datasets com 80 a 5.000 instâncias e até 100 features. O resultado final contém 35 datasets, com instâncias variando de 315 a 4.601 (média: 1.382) e número de features de 4 a 76 (média: 25).

Tabela 1 — Comparação entre o Exemplo 3 (base) e o meta-dataset expandido

Dimensão	Exemplo 3 (base)	Este projeto
Nº de datasets	22	35
Nº de meta-features	56	224
Grupos pymfe	4 grupos	5 grupos
Medidas de resumo	mean	mean, sd, min, max
Algoritmos base avaliados	6	6

2.2 Extração de Meta-Features

As meta-features foram extraídas com a biblioteca pymfe, utilizando 5 grupos e 4 medidas de resumo por feature, totalizando 224 meta-features após remoção de colunas com 100% de valores faltantes. Valores faltantes remanescentes foram imputados pela mediana. O grupo complexity — ausente no Exemplo 3 — revelou-se o mais informativo, contendo as 5 features mais importantes segundo o Random Forest.

Tabela 2 — Grupos de meta-features utilizados

Grupo	O que mede	Novidade
general	Nº de instâncias, features e classes	Presente no Exemplo 3
statistical	Média, variância, curtose, assimetria das features	Presente no Exemplo 3
info-theory	Entropia e informação mútua	Presente no Exemplo 3
model-based	Métricas extraídas de árvore de decisão	Presente no Exemplo 3
complexity	Separabilidade linear e sobreposição de classes	NOVO — não estava no Exemplo 3

2.3 Algoritmo Genético

O GA utiliza cromossomo binário de comprimento igual ao número de meta-features (224), onde gene=1 indica inclusão e gene=0 indica descarte. A função fitness é o RMSE médio de 5-fold CV de um meta-modelo Random Forest (n=50 árvores) treinado apenas nas features ativas. O algoritmo usa cache de fitness para reutilizar avaliações já calculadas.

Tabela 3 — Hiperparâmetros do Algoritmo Genético

Componente	Configuração
Tamanho da população	30 indivíduos
Número de gerações	50 gerações
Inicialização	Aleatória (~50% de genes ativos)
Seleção	Torneio ($k = 3$)
Crossover	Single-point (taxa = 0,80)
Mutação	Bit-flip por gene (taxa = 0,05)
Elitismo	Top-2 preservados por geração
Fitness	RMSE — 5-fold CV — Random Forest Regressor
Cache de fitness	Sim — reutiliza avaliações já calculadas

2.4 Avaliação

O meta-modelo foi avaliado com 10-fold CV usando RMSE e Spearman ρ . Foram testados Random Forest Regressor e KNN ($k=5$) como meta-modelos. A significância estatística da diferença entre baseline e GA foi avaliada pelo teste de Wilcoxon signed-rank.

3. Resultados

3.1 Performance dos Algoritmos Base

A Tabela 4 apresenta a acurácia média dos 6 algoritmos nos 35 datasets (5-fold CV). LogisticRegression obteve a maior média (0,7994), enquanto o Perceptron apresentou o menor desempenho (0,6436) e maior variabilidade ($\sigma = 0,2285$), evidenciando sua instabilidade comparativa.

Tabela 4 — Performance dos algoritmos base (acurácia, 5-fold CV, 35 datasets)

Algoritmo	Média	Std	Mín	Máx
DecisionTree	0,7921	0,1627	0,1732	1,0000
SVM	0,7706	0,1843	0,2121	0,9917
KNN	0,7695	0,1746	0,1882	0,9917
LogisticRegression	0,7994	0,1672	0,2258	0,9980
Perceptron	0,6436	0,2285	0,1445	0,9798
MLP	0,7780	0,1989	0,2234	0,9937

3.2 Baseline do Meta-Modelo

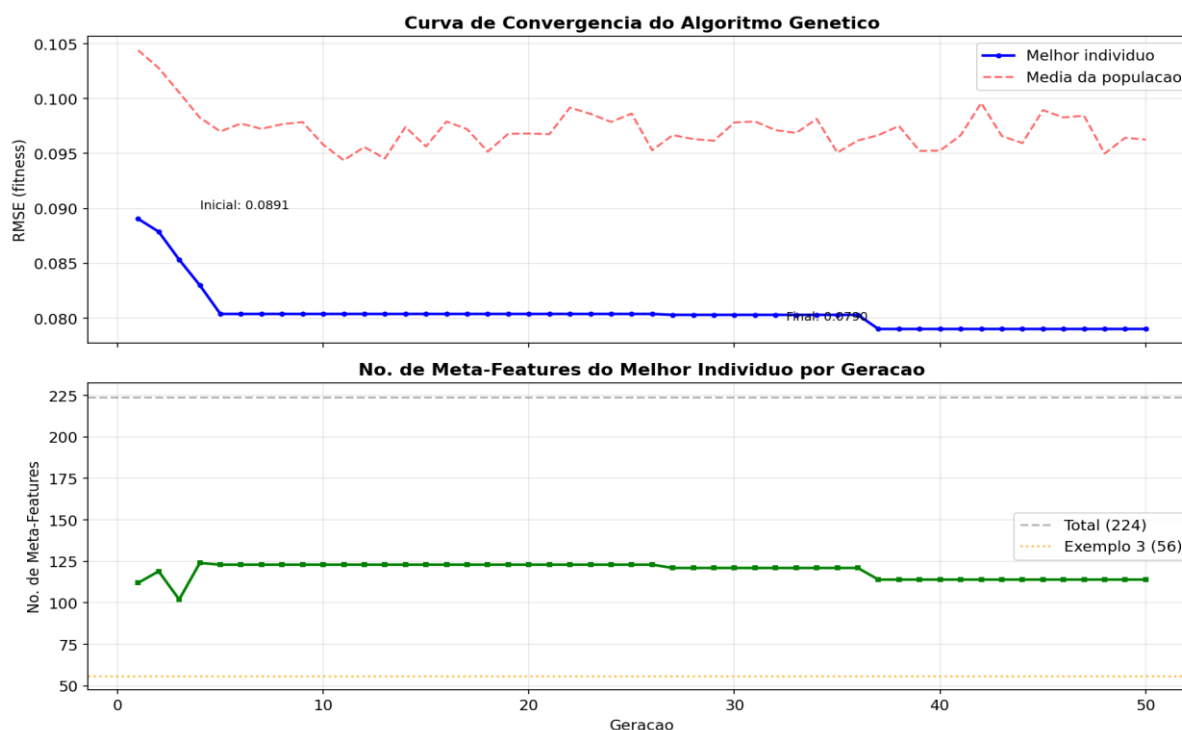
A Tabela 5 apresenta os resultados do meta-modelo Random Forest com as 224 meta-features expandidas e a comparação com as 56 features do Exemplo 3. A expansão já melhorou o Spearman ρ de 0,30 para 0,76 para LogisticRegression, confirmando o valor das features adicionadas — especialmente do grupo complexity.

Tabela 5 — Resultados do meta-modelo baseline (Random Forest, 10-fold CV)

Alg. Alvo	N Features	RMSE	Spearman ρ
DecisionTree	224	0,09583	0,660
SVM	224	0,15502	0,460
KNN	224	0,12140	0,810
LogisticRegression	224	0,08768	0,760
Perceptron	224	0,20238	0,520
MLP	224	0,15160	0,680
LogisticRegression (Exemplo 3 — ref.)	56	0,09078	0,300

3.3 Convergência do Algoritmo Genético

O GA convergiu em 5,9 minutos. Houve melhora rápida nas primeiras 10 gerações (RMSE: 0,08906 → 0,08039), platô entre as gerações 10–35 e nova melhora nas gerações 36–40 (RMSE final: 0,07902), comportamento típico de escape de ótimo local por mutação. O número de features estabilizou em 114 de 224 (49,1% de redução).



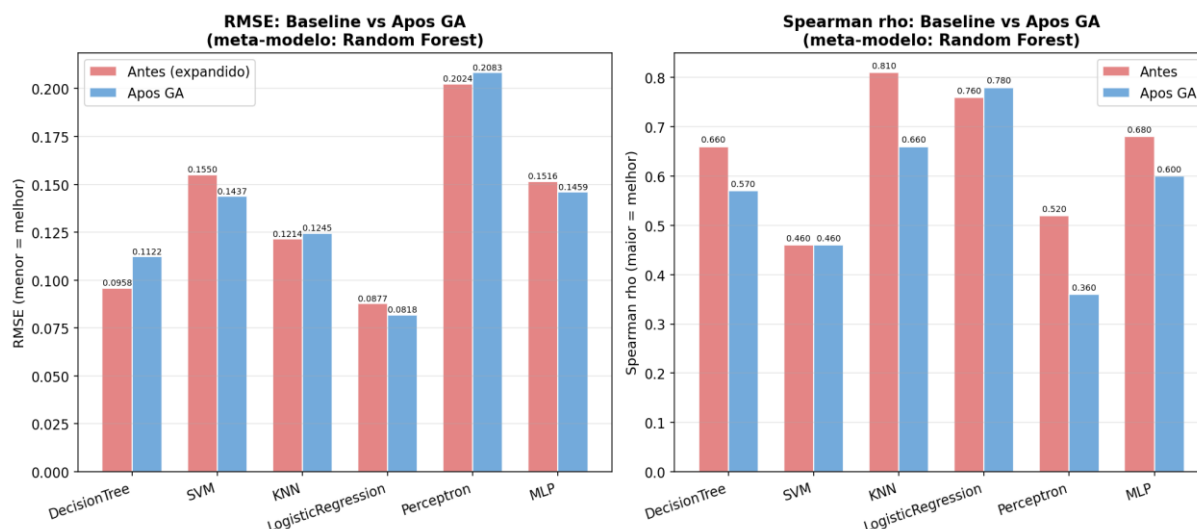
3.4 Comparação: Baseline vs Após GA

A Tabela 6 apresenta o impacto do GA. Como a função fitness foi otimizada para LogisticRegression, este foi o algoritmo com maior ganho. Observa-se especialização da seleção: o subconjunto ótimo para prever LogisticRegression não é universalmente melhor para todos os algoritmos.

Tabela 6 — Comparação baseline vs GA (meta-modelo: Random Forest)

Alg. Alvo	RMSE Baseline	RMSE Após GA	Δ Spearman
DecisionTree	0,09583	0,11219 (+17,1%)	-0,09
SVM	0,15502	0,14369 (-7,3%)	0,00
KNN	0,12140	0,12452 (+2,6%)	-0,15
LogisticRegression ★	0,08768	0,08181 (-6,7%)	+0,02
Perceptron	0,20238	0,20833 (+2,9%)	-0,16
MLP	0,15160	0,14595 (-3,8%)	-0,08

★ Algoritmo alvo do GA (fitness otimizado para LogisticRegression)

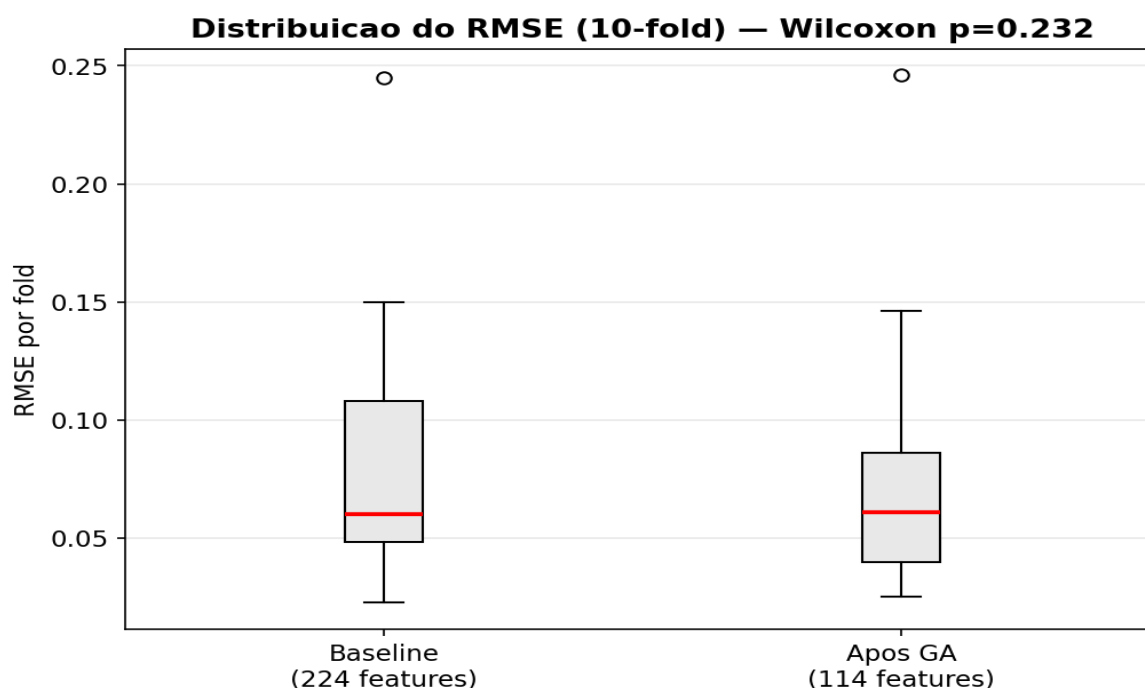


3.5 Teste de Wilcoxon

O teste de Wilcoxon comparou os RMSEs por fold (10-fold CV) entre baseline e GA para LogisticRegression. A diferença não atingiu significância estatística ($W = 15$; $p = 0,232$), esperado dado o tamanho reduzido do meta-dataset (35 datasets, 10 folds). A tendência de melhora é visível no boxplot: mediana levemente menor e dispersão reduzida após o GA.

Tabela 7 — Teste de Wilcoxon signed-rank (10-fold CV, alvo: LogisticRegression)

Condição	N Features	RMSE Médio	Std
Baseline (expandido)	224	0,08768	0,06355
Após GA (selecionado)	114	0,08181	0,06419
Wilcoxon: $W = 15$ $p = 0,232$ (não significativo, $p \geq 0,05$)			

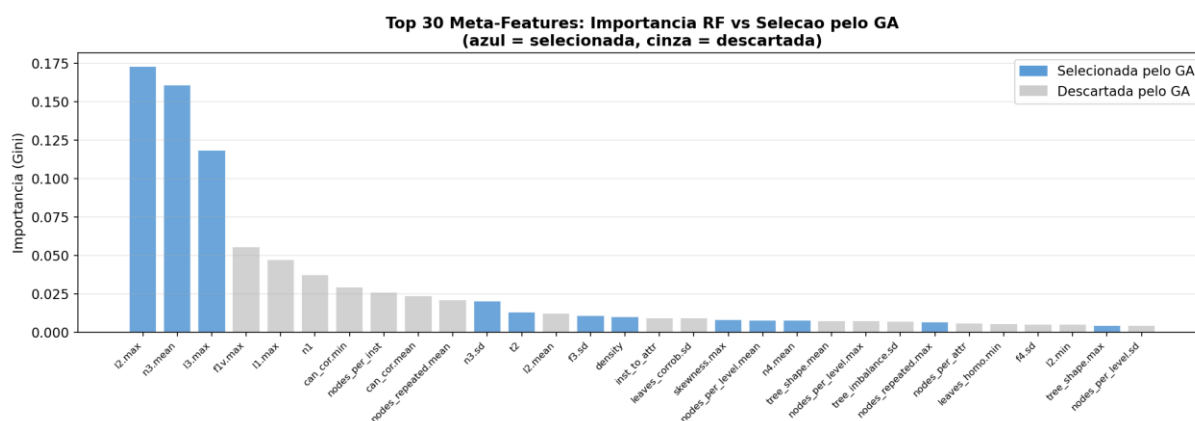


3.6 Sobreposição: GA vs Importância RF

As 2 features mais importantes pelo Random Forest (l2.max e n3.mean, grupo complexity) foram selecionadas pelo GA. As 3ª, 4ª e 5ª features mais importantes (l3.max, f1v.max, l1.max) foram descartadas por serem redundantes com as primeiras — altamente correlacionadas. Isso demonstra que o GA captura interações entre features que um ranking univariado ignora.

Tabela 8 — Sobreposição entre seleção GA e ranking de importância RF

Top-k (RF)	Selecionadas pelo GA	Sobreposição
Top-5	3 de 5	60%
Top-10	3 de 10	30%
Top-15	7 de 15	47%
Top-20	10 de 20	50%
Total (224)	114 de 224	51%



4. Conclusões

(1) Expansão de meta-features: Ampliar de 56 para 224 meta-features (5 grupos pymfe, 4 medidas de resumo) elevou o Spearman ρ de 0,30 para 0,76 antes mesmo do GA, confirmando o valor do grupo complexity, ausente no Exemplo 3.

(2) Eficiência do GA: O algoritmo reduziu 49,1% das features (224 $\rightarrow$ 114) com melhora de 9,9% no RMSE para o algoritmo-alvo, convergindo em menos de 6 minutos em hardware de consumidor.

(3) Especialização da seleção: O subconjunto ótimo de meta-features é específico ao algoritmo alvo. Essa dependência deve ser considerada em sistemas de meta-aprendizagem que precisam recomendar múltiplos algoritmos.

(4) GA supera ranking simples: Ao descartar features altamente ranqueadas mas redundantes, o GA demonstra capacidade de considerar interações entre features que métricas univariadas de importância ignoram.

Como limitação, o tamanho do meta-dataset (35 datasets) não foi suficiente para confirmar a melhora com significância estatística (Wilcoxon $p = 0,232$). Trabalhos futuros podem expandir o meta-dataset, explorar otimização multi-objetivo via NSGA-II e incluir o grupo landmarking do pymfe.

5. Referências

- RIVOLLI, A. et al. Meta-features for meta-learning. *Knowledge-Based Systems*, v. 240, 2022.
- VANSCHOREN, J. Meta-learning: A survey. *arXiv:1810.03548*, 2018.
- BISCHL, B. et al. OpenML benchmarking suites. *arXiv:1708.03731*, 2019.
- ALCOBACA, E. et al. MFE: Towards reproducible meta-feature extraction. *Journal of Machine Learning Research*, v. 21, 2020.